TECHNICAL GUIDE | KNOWLEDGE BASE

Local RAG Assistant

A practical reference for document ingestion, vector retrieval, grounded answers, and operating this example application.

> **What this guide covers**
> How the sample app handles a question from PDF ingestion through answer generation, why local retrieval is useful, and how to diagnose common quality problems.

Retrieval-Augmented Generation (RAG)

RAG combines search with a language model. The application first finds passages in a document collection, then gives those passages to a model as evidence for an answer. This lets the system answer questions about private or specialized material that may not be in the model's training data.

Why use local retrieval?

- Keep the document store and vector search on infrastructure you control.
- Search internal policies, runbooks, contracts, technical manuals, and research notes.
- Return answers with source passages so readers can check the evidence.
- Reduce repeated manual searching and, when models are self-hosted, avoid per-request model charges.

> **Privacy depends on every component.** This sample computes embeddings locally and stores documents in PostgreSQL, but its current answer-generation step calls Google's Gemini API. Retrieved text and the question are sent to that service. Use a locally hosted language model if the complete workflow must stay on private infrastructure.

Request flow in this project

PDF -> text chunks -> BGE embeddings -> PostgreSQL + pgvector
Top passages -> Gemini answer -> Streamlit display

The main components are the ingestion script, FastAPI endpoint, PostgreSQL database with pgvector, Sentence Transformers, Gemini, and the Streamlit interface.

1. Ingesting documents

Ingestion turns files into searchable passages. The stored text and metadata determine what the assistant can later retrieve.

Current PDF pipeline

1. **Extract:** PyMuPDF reads selectable text from each PDF page. The current helper joins page text into one string; it does not perform OCR or preserve page numbers.
2. **Split:** LangChain's RecursiveCharacterTextSplitter creates chunks of about 400 characters with 50 characters of overlap. It prefers paragraph breaks, line breaks, sentence separators, then spaces.
3. **Embed:** BAAI/bge-small-en-v1.5 converts each passage to a 384-dimensional vector.
4. **Store:** PostgreSQL stores the file path, chunk index, text, and vector in the documents table.

Chunking for useful evidence

A chunk should contain a complete idea, including the heading that introduces it. If a heading is left at the end of one chunk and its explanation begins in the next, a search may find the heading without enough evidence to answer. Keep sections together where possible, use overlap to carry context across boundaries, and inspect real extracted chunks before tuning sizes.

Very small chunks can lose context. Very large chunks can dilute search relevance and use more prompt space. There is no universal best size: compare retrieval results on representative questions from the actual corpus.

PDF extraction limits

- Scanned pages need OCR before they can be searched as text.
- Tables, columns, headers, and footers may extract in an unexpected order. Check the extracted text for important PDFs.
- This sample stores source path and chunk index, but not page number or section heading as separate fields.
- Re-ingesting the same path replaces its old chunks atomically in the sample script.

> **Practical check:** Before debugging the language model, confirm that the expected wording is present in the stored chunk text. If the words never made it through extraction, retrieval cannot find them.

2. Embeddings and vector search

Embeddings represent text as vectors. Search ranks passages by vector distance, then returns the closest candidates as context.

Embedding model in this app

The project uses **BAAI/bge-small-en-v1.5**, which produces 384-dimensional embeddings compatible with the database column `VECTOR(384)`. Passage text is embedded without an instruction. Search questions use the model's recommended retrieval prefix:

```
Represent this sentence for searching relevant passages:
```

Use the same model and compatible settings for questions and stored passages. If you change the embedding model or its passage representation, re-embed the existing documents before comparing new query vectors with them. Changing only the query prefix does not require re-embedding passages.

pgvector retrieval

The retriever orders rows by the pgvector cosine-distance operator `<=>`. It returns the five nearest chunks, along with a displayed similarity computed as `1 - cosine distance`. The application keeps the ranked results rather than applying a universal 0.70 cutoff; score ranges vary by model and corpus, so an uncalibrated cutoff can hide useful evidence.

Index notes

The sample schema defines an IVFFlat cosine index. PostgreSQL maintains indexes as rows are inserted or updated. Adding chunks does not by itself require manually reloading the index. Re-embedding documents changes stored vectors, while index creation and tuning are separate database operations. Validate retrieval quality after changes, especially with approximate indexes and small collections.

> **Read scores as a ranking aid.** A higher score means a closer vector match among candidates; it does not prove that a passage answers the question. Inspect the returned passage text and tune any score cutoff against a labeled set of queries.

Retrieval improvements

When nearest passages are not precise enough, evaluate section-aware chunking, lexical-plus-vector hybrid search, query expansion, or a cross-encoder reranker. Add one change at a time and compare against a fixed set of questions and relevant passages.

3. Grounded answers and sources

The language model should explain evidence found by retrieval, not fill gaps with plausible outside knowledge.

Answer generation

The API labels retrieved passages as `[S1]`, `[S2]`, and so on, then includes them with the user's question. The prompt asks Gemini to answer only from that context, ignore instructions contained inside document text, and cite factual claims with source labels. If no passages are retrieved, the API returns: *I don't know based on the provided documents.*

What a citation means

A citation points to a retrieved chunk, not necessarily the entire source file. Open the matching source panel to inspect the complete chunk, its file path, chunk index, and similarity score. Check that each important statement is actually supported by the cited text.

Common answer failures

- **Relevant fact missing:** retrieval selected the wrong chunks, or the answer is split across chunk boundaries.
- **Correct fact with no source:** prompt instructions or response formatting may be insufficient; require citations and inspect the generated response.
- **Unsupported detail:** tighten grounding instructions and reduce irrelevant context. Do not interpret a high similarity score as proof.
- Answer says "I don't know": inspect source panels. If the evidence is absent, fix ingestion or retrieval; if present, review how context is assembled and prompted.

Prompt injection in documents

Retrieved text is data, even if it contains phrases that look like instructions. The prompt explicitly tells the model to ignore document instructions. For sensitive deployments, add stronger isolation, output checks, and access controls; prompt wording alone is not a security boundary.

> **External model note:** Because this project uses Gemini, the question and retrieved passages leave the local database when an answer is generated. Review the provider's terms and your organization's data rules before sending confidential text.

4. Running the sample application

The repository separates ingestion, API serving, retrieval, answer generation, and the browser interface.

Components

- **scripts/ingest_pdf.py:** reads one PDF, chunks it, creates embeddings, and replaces that file's stored chunks.
- **app/api.py:** exposes `/health` and `/chat`.
- **app/retriever.py:** queries PostgreSQL with pgvector and returns ranked source chunks.
- **app/llm.py:** sends retrieved context to Gemini and returns the generated text.
- **ui/streamlit_app.py:** submits questions and displays the answer and expandable source panels.

Typical local startup

```
docker compose up -d
python -m scripts.ingest_pdf data/sample.pdf
uvicorn app.api:app --reload
streamlit run ui/streamlit_app.py
```

The database environment variables are `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`. Gemini requires `GOOGLE_API_KEY`; `GEMINI_MODEL` selects the model and defaults to `gemini-2.5-flash`. Keep credentials in the ignored `.env` file and never place secret values in documents or logs.

Health and errors

Check `GET /health` to confirm that FastAPI is responding. A blank question is rejected. Gemini failures are returned as HTTP 502, and the Streamlit interface displays request errors instead of treating them as answers. A database failure appears before generation and should be checked in the API terminal.

Operational boundaries

Embedding runs locally through Sentence Transformers. PostgreSQL stores the document chunks and vectors. Gemini generation is remote. The sample has no authentication or per-user document access rules, so do not expose it to untrusted networks without adding those controls.

5. Troubleshooting and evaluation

Debug the pipeline from left to right: extraction, chunking, embedding, retrieval, prompt construction, then generation.

Troubleshooting checklist

- **No useful sources:** verify the PDF path and that extracted text contains the answer; inspect chunk boundaries; confirm the query embedding uses the configured model; inspect the ranked passages and scores.
- "I don't know" despite relevant evidence: expand the retrieved passages and check that the complete supporting sentence reaches the prompt. Look for a heading or explanation split into adjacent chunks.
- **Vector dimension error:** the embedding size must match the database column. This sample expects 384 dimensions.
- **Vector cast or insert error:** confirm vector values are serialized in pgvector text format and cast to `vector`.
- **PostgreSQL connection error:** verify the database container is running, connection settings are correct, and the vector extension and documents table exist.
- **Gemini error:** check the API key, model setting, network access, and provider response in the API logs.
- **Duplicate passages:** ingest using the same canonical file path so prior chunks for that source are replaced.

Evaluate with real questions

Maintain a small set of questions with expected supporting passages and acceptable answers. For each run, record whether the right passage appears in the top results, whether the answer is supported, whether citations point to the right chunk, and whether the system correctly abstains when evidence is missing.

Useful retrieval measures include precision at k, recall at k, and mean reciprocal rank. Answer checks should measure correctness, faithfulness to retrieved text, and citation accuracy. Review examples manually; aggregate scores can hide a broken chunk or a sensitive failure.

Local RAG use cases

Internal documentation search, database runbook assistance, contract review, research-paper discovery, customer-support knowledge bases, and secure enterprise search are common patterns. Use access controls and document-level permissions when different users are allowed to see different material.

> **Improvement loop:** collect failure examples, identify which stage caused each one, change one part of the pipeline, then rerun the same questions. This makes retrieval and answer quality changes measurable.

6. Design and operating tradeoffs

Model, infrastructure, and operational choices affect quality, cost, latency, and privacy. Measure them against your users' real questions and workload.

Embedding and vector-store options

The sample uses BGE small English embeddings and PostgreSQL with pgvector. Other embedding models may suit different languages, domains, or hardware, but vectors from different models are not interchangeable. Re-embed passages when switching models. ChromaDB is often used for local prototypes; FAISS provides a vector-search library; PostgreSQL with pgvector combines vector search with relational data and SQL. Choose based on persistence, filtering needs, operations, scale, and team experience.

Performance and capacity

- Batch embedding work during ingestion and cache embeddings when the source text has not changed.
- Measure PDF parsing, embedding, database search, and model generation separately to find the actual latency bottleneck.
- GPU acceleration can reduce embedding and model latency when supported by the selected hardware and runtime.
- Quantized language models can reduce memory use. Hardware requirements depend on model size, quantization, context length, concurrency, and inference backend; benchmark the intended workload.

Cost and business tradeoffs

Self-hosted generation can avoid per-token provider charges, but it adds hardware, maintenance, and monitoring costs. A hosted model such as the Gemini API is simpler to integrate but may incur usage charges and sends prompts and retrieved passages to the provider. Faster access to internal knowledge can reduce repeated support work and help teams use existing documentation, provided the retrieved information is current and access is controlled.

Advanced retrieval patterns

- **Hybrid search:** combine keyword matching with vector similarity when exact names, codes, or identifiers matter.
- **Reranking:** use a cross-encoder to reorder a wider candidate list before sending the best passages to the answer model.
- **Query expansion or multi-query search:** generate alternative search phrasings when users describe the same concept in different words.
- **Caching and conversation memory:** reduce repeated work while keeping each request tied to the right user, document permissions, and current context.
- **Tool-using workflows:** allow the model to call systems only with explicit authorization, scoped credentials, and audit logs.

Monitoring and continuous improvement

Track ingestion throughput, extraction failures, embedding latency, retrieval relevance, answer latency, abstentions, citation accuracy, and user feedback. Store only the diagnostic information allowed by your privacy policy. Review representative failures and improve chunking, retrieval, or prompting one step at a time.

> **Operating principle:** document search quality is an end-to-end property. Better model generation cannot compensate for missing text, broken chunks, stale vectors, or retrieval that omitted the supporting passage.